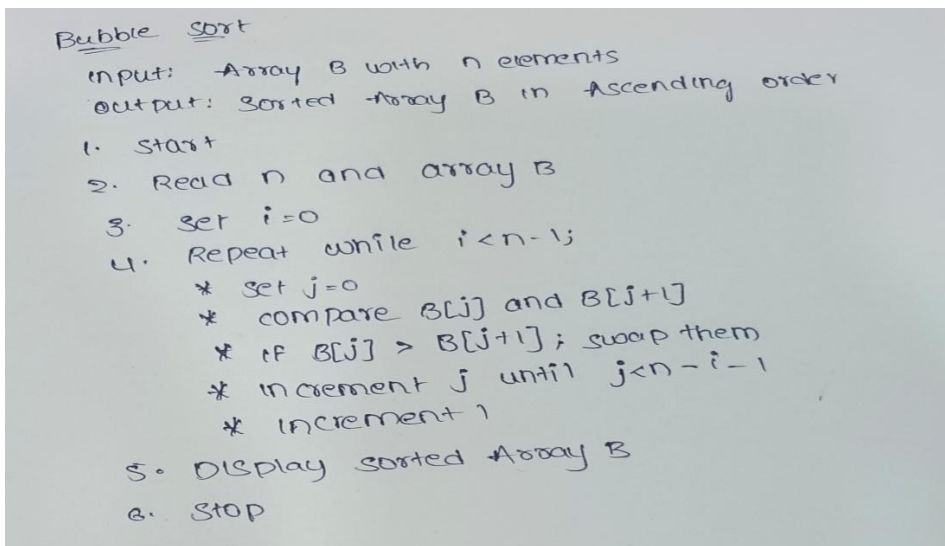


PRACTICAL: 01

Aim: To implement and find the time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

Bubble Sort:

Algorithm:



Bubble sort
input: Array B with n elements
output: Sorted Array B in Ascending order

1. Start
2. Read n and array B
3. Set $i=0$
4. Repeat while $i < n-1$
 - * Set $j=0$
 - * compare $B[j]$ and $B[j+1]$
 - * if $B[j] > B[j+1]$; swap them
 - * increment j until $j < n-i-1$
 - * increment i
5. Display sorted Array B
6. Stop

B)

Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

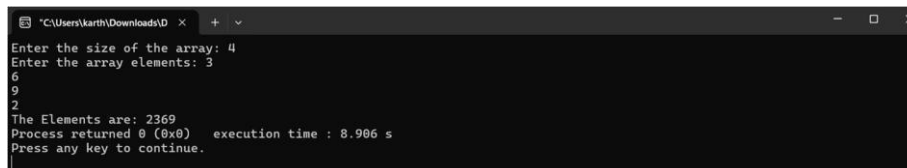
    for(i=0; i<size; i++){
```

```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i];
    }

    return 0;
}
```

Output:



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\karth\Downloads\...". The prompt displays the following text: "Enter the size of the array: 4", "Enter the array elements: 3", followed by the numbers "6", "9", and "2" on separate lines. Then it says "The Elements are: 2369". Below that, it shows "Process returned 0 (0x0) execution time : 8.906 s" and "Press any key to continue.".

Time Complexity:

logic 4 Time complexity

Bubble sort:

```

for (i=0; i<n-1; i++) {
    for (j=0; j<n-i-1; j++) {
        if (a[j] > a[j+1]) {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}

```

$T(n) = (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = n^2 - n$
 $T(n) = O(n^2)$

→ Best case = Average case = worst case = $O(n^2)$

Bubble sort

i) Best case:
The array is sorted
no. of comparison = $n-1$
 $T(n) = n-1$
 $T(n) = O(n)$

ii) Average case:
 $(n-1) + (n-2) + (n-3) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

iii) Worst case:
Array is in reverse order
 $(n-1) + (n-2) + (n-3) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

Insertion Sort: Algorithm:

insertion sort:

Input: Array A with n elements
Output: Sorted Array A in Ascending order

1. Start
2. Read n and Array A
3. Set $i=1$
4. Set Key = $A[i]$ and $j=i-1$
5. while $j>0$ and $A[j]>Key$, shift $A[j]$ to $A[j+1]$
6. Insert Key at $A[j+1]$
7. Increment i and repeat steps 4-6 until $i<n$
8. Display Sorted Array A
9. Stop.

Program / Code:

```
#include<iostream>

using namespace std;

int main(){
    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){
        cin>>a[i];
    }

    for(i=0; i<size-1; i++){
        min = i;

        for(j=i+1; j<size; j++){
            if(a[j] < a[min]){
                min = j;
            }
        }

        if(min != i){
            temp = a[i];
            a[i] = a[min];
            a[min] = temp;
        }
    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){
        cout<<a[i]<<" ";
    }
}
```

```

    }

    return 0;

}

```

Output:

```

C:\Users\karth\Downloads\D >
Enter the size of the array: 4
Enter the array elements: 6
3
9
2
The Elements are: 2 3 6 9
Process returned 0 (0x0)   execution time : 19.363 s
Press any key to continue.

```

Time Complexity:

insertion sort:

```

for (i=1; i<n; i++) {
    key = arr[i]
    j = i-1
    while (j >= 0 & arr[j] > key) {
        arr[j+1] = arr[j]
        j--
    }
    arr[j+1] = key;
}

```

$T(n) = n+1 + (1+1+1+1+1)$
 $T(n) = n+6$
 $T(n) = O(n) \rightarrow$ Best case

worst case

i	j	1+2+...+n
1	1	$T(n) = \frac{n(n+1)}{2}$
2	2	$= \frac{n^2-n}{2}$
3	3	
...	...	
n-1	n-1	

$T(n) = O(n^2)$

insertion sort

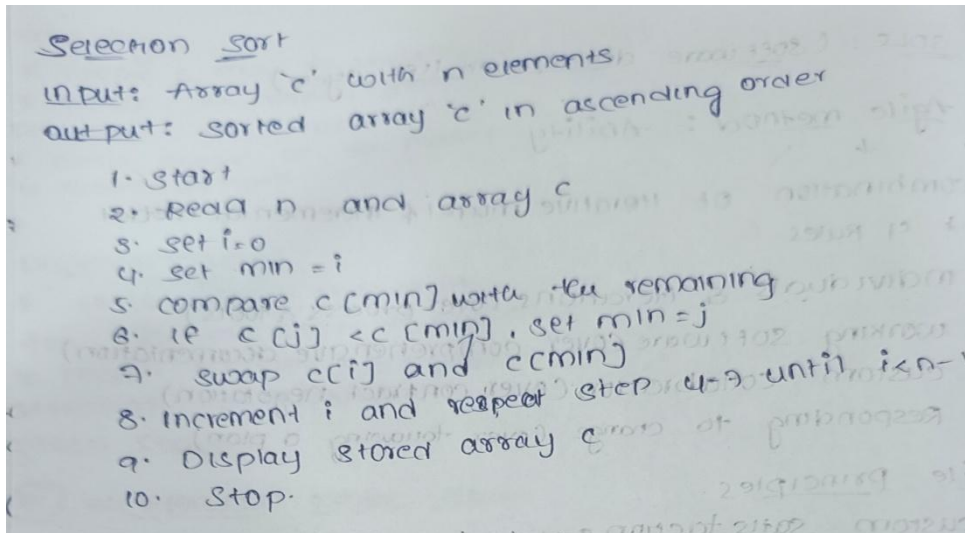
i) Best case:
 Array is already sorted one comparison
 $n-1$
 $T(n) = n-1$
 $T(n) = O(n)$

ii) Average case:
 $1+2+3+\dots+(n-1)$
 $T(n) = \frac{n(n-1)}{2}$
 $= \frac{(n^2-n)}{2}$
 $T(n) = O(n^2)$

iii) Worst case:
 $(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

Selection Sort:

Algorithm:



Program / Code:

```
#include <iostream>

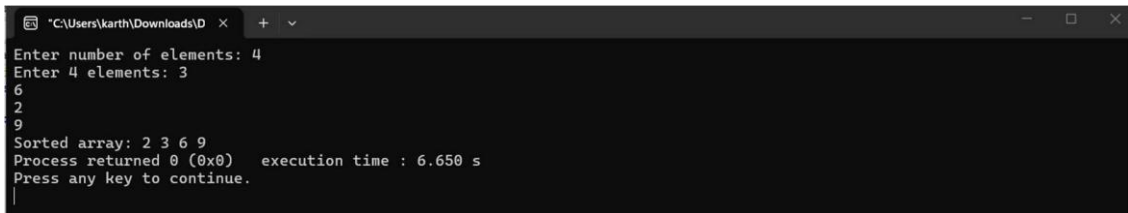
using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
```

```
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    selectionSort(arr, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\D x + v
Enter number of elements: 4
Enter 4 elements: 3
6
2
9
Sorted array: 2 3 6 9
Process returned 0 (0x0)   execution time : 6.650 s
Press any key to continue.
```

Time complexity:

Selection Sort:

```

for (i=0; i<n-1; i++) {
    min = i;
    for (j=i+1; j<n; j++) {
        if (arr[j] < arr[min]) {
            min = j;
        }
    }
    temp = arr[min];
    arr[min] = arr[i];
    arr[i] = temp;
}
    
```

$$T(n) = (n-1) + (n-2) + \dots + 1 + 1$$

$$T(n) = \frac{n(n-1)}{2} + \frac{n}{2}$$

$$T(n) = \frac{n^2 - n}{2} + \frac{n}{2} + 1$$

$$= \frac{n^2 - n + n + 2}{2}$$

$$= \frac{n^2 + 2}{2}$$

$$= \frac{n^2}{2} + 1$$

$$T(n) = O(n^2)$$

Selection Sort:

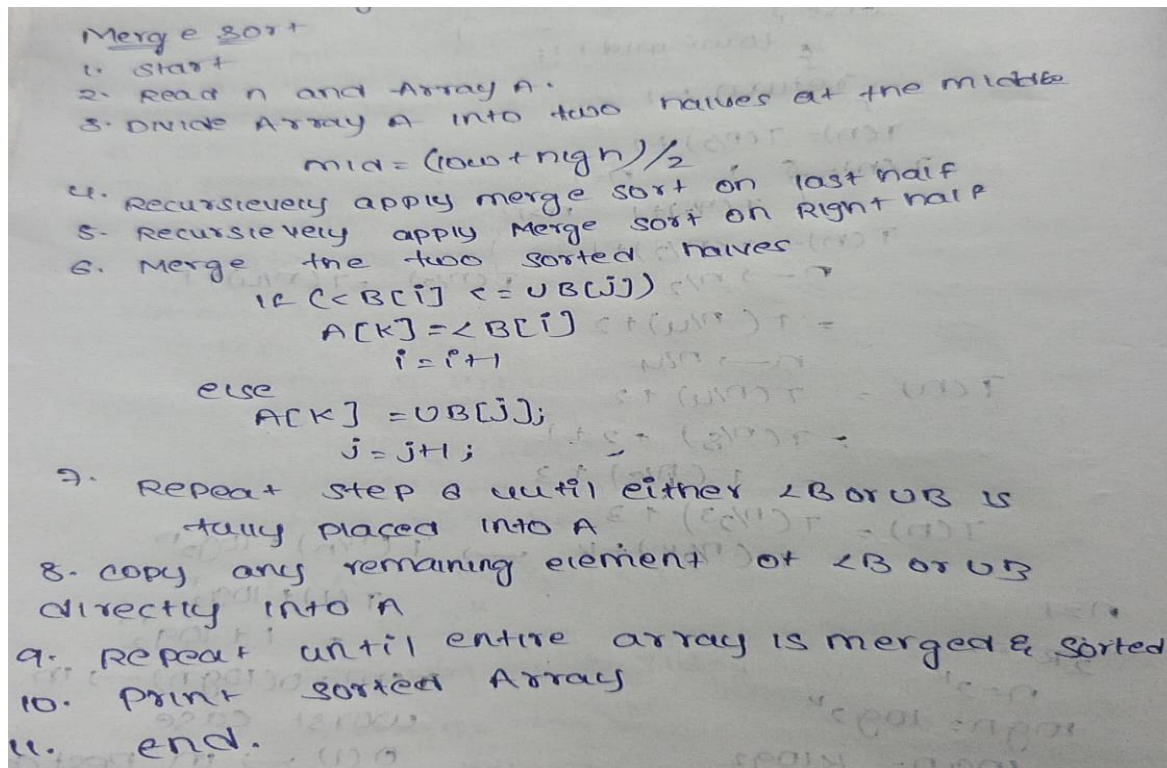
i) Best case:
Perform all comparison
 $(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $= \frac{(n^2 - n)}{2}$
 $T(n) = O(n^2)$

ii) Average case
Check every pass
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

iii) Worst case:
All comparison
 $T(n) = O(n^2)$

A) Merge Sort:

Algorithm:



Program / Code:

```
#include <iostream>

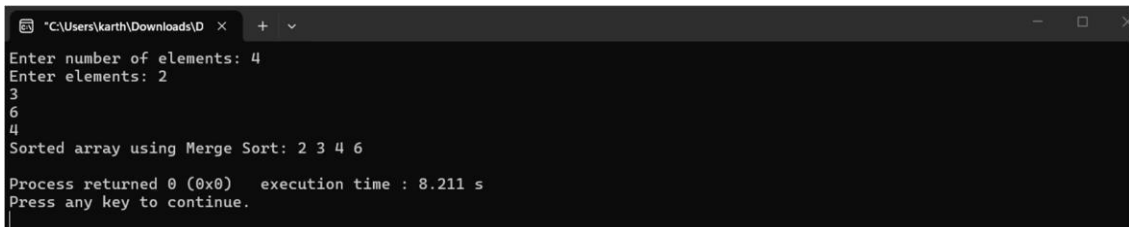
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
```

```
    }  
    for (int j = 0; j < n2; j++) {  
        R[j] = arr[mid + 1 + j];  
    }  
    int i = 0;  
    int j = 0;  
    int k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while (i < n1) {  
        arr[k++] = L[i++];  
    }  
    while (j < n2) {  
        arr[k++] = R[j++];  
    }  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

```
    }  
}  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array using Merge Sort: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\D > + v  
Enter number of elements: 4  
Enter elements: 2  
3  
6  
4  
Sorted array using Merge Sort: 2 3 4 6  
Process returned 0 (0x0)   execution time : 8.211 s  
Press any key to continue.
```

Time Complexity:

Merge sort:

No. of divisions: $\log_2 n$ ($\log_2 n$)

merge: each level merges all n elements

total time: $T(n) = 2T(n/2) + n$ — (1)

$n \rightarrow n/2$ put in (1)

$T(n/2) = 2T(n/4) + n/2$ — (2)

$T(n) = 2(2T(n/4) + n/2) + n$ — (3)

$T(n) = 2^2 T(n/4) + 2n$ — (3)

$n \rightarrow n/4$ put in (3)

$T(n/4) = 2T(n/8) + n/4$ — (4)

$T(n) = 2^3 T(n/8) + 3n$ — (4)

$T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1 \quad n = 2^k$

Apply log on both sides

$\log n = \log 2^k$

$k = \log n$

$T(n) = 2^{\log n} T\left(\frac{n}{2^{\log n}}\right) + \log n \cdot n$

$= n^{\log 2} + \frac{n}{n^{\log 2}} + \log n(n)$

$= n \log n + n$

$T(n) = O(n \log n)$

Merge sort:

i) Best case:

Divides Array into two halves

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

$T(n) = O(n \log n)$

ii) Average case:

$n + n/2 + n/4 + \dots + 1$

$T(n) = n \log n$

$T(n) = O(n \log n)$

iii) Worst case:

Merge sort continuously divides the array into halves. Then merges all elements

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

$T(n) = n + \log_2 n$

$T(n) = O(n \log n)$

B) Quick Sort: Algorithm:

Quick sort:

1. Start
2. Read n and Array A
3. Choose pivot as the first element of the current sub-array
 $\text{pivot} = A[\text{low}]$
4. Move i from left and j from right, scanning toward each other
 while $[A[i]] \leq \text{pivot}$ and $i < \text{high}$
 $i = i + 1$
 while $[A[j]] \geq \text{pivot}$ and $j > \text{low}$
 $j = j - 1$
5. If $i < j$
 swap $[A[i], A[j]]$
6. Repeat step 4 until i and j cross ($i \geq j$)
7. Place pivot in its correct sorted position
 swap $(A[\text{low}], A[i])$
 return i ; // pivot final index
8. Recursively apply Quick sort on left part
9. Recursively apply Quick sort on right part
10. Repeat until each sub-array has one or zero element.
11. Print sorted array
12. End.

Program / Code:

```
#include <iostream>

using namespace std;

int partition(int arr[], int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
```

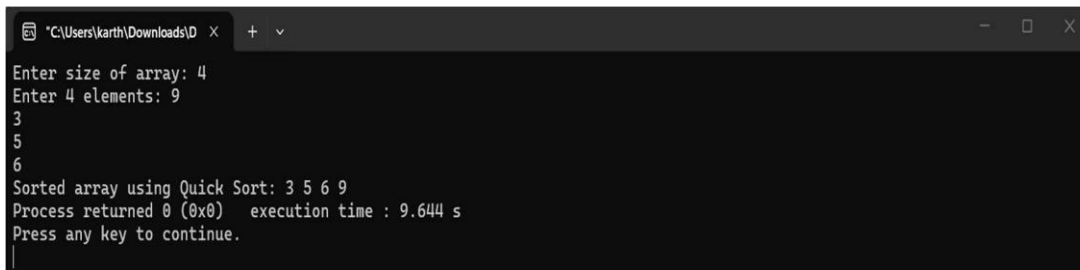
```
int end = ub;
while (start < end) {
    while (arr[start] <= pivot) {
        start++;
    }
    while (arr[end] > pivot) {
        end--;
    }
    if (start < end) {
        swap(arr[start], arr[end]);
    }
}
swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1);
        quickSort(arr, loc + 1, ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
```

```
cout << "Enter " << n << " elements: ";  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
quickSort(arr, 0, n - 1);  
cout << "Sorted array using Quick Sort: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
return 0;  
}
```

Output:



```
*C:\Users\karth\Downloads\D x + v  
Enter size of array: 4  
Enter 4 elements: 9  
3  
5  
6  
Sorted array using Quick Sort: 3 5 6 9  
Process returned 0 (0x0) execution time : 9.644 s  
Press any key to continue.  
|
```

Time Complexity:

Quick Sort:
The pivot divides this array into two equal halves
→ $2T(n/2)$
for sorting we use elements at last (or)
 $T(n) = 2T(n/2) + n$ — (1)
 $n \rightarrow \frac{n}{2}$ put in (1)
 $T(n/2) = 2T(n/4) + n/2$
 $T(n) = 2T(n/4) + n/2$ — (2)
sub (2) in (1)
 $T(n) = 2(2T(n/4) + n/2) + n$
 $T(n) = 2^2T(n/4) + 2n$ — (3)
 $n \rightarrow \frac{n}{4} = 2^2T(\frac{n}{4}) + 2(\frac{n}{4})$ — (4)
sub (4) in (3)
 $T(n) = 2^2[2^2T(\frac{n}{16}) + 2(\frac{n}{4})] + 2n$
 $= 2^4T(\frac{n}{16}) + 4n$
 $T(n) = 2^kT(\frac{n}{2^k}) + kn$
 $T(n) = 2^kT(\frac{n}{2^k}) + kn$
 $\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \text{Apply log on both sides}$
 $\log n = \log 2^k \Rightarrow k = \log n$
 $T(n) = 2^{\log n} T(\frac{n}{2^{\log n}}) + \log n \cdot n$
 $= n \log^2 n + \frac{n}{n} \log n$
 $= n \log n + n$
 $T(n) = O(n \log n)$

Quick Sort:

- Best case:
Pivot divides the array into two equal parts
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 + n/4 + n/4 + \dots + 1$
 $T(n) = n \log_2 n$
 $T(n) = O(n \log n)$
- Average case: Divides the balance parts
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4$
 $T(n) = n \log_2 n$
 $T(n) = O(n \log n)$
- Worst case:
 $n + (n-1) \rightarrow (n-2) + (n-3) + \dots + 1$
 $T(n) = n(n-1)/2$
 $= (n^2 - n)/2$
 $T(n) = O(n^2)$